Report for Laboratory 8: Motor Position Control

Julian Soh

Department of Mechanical Engineering, Saint Martin's University

ME/EE 477—Embedded Real-Time Computing for Mechanical Engineers

July 19, 2026

Abstract. This lab implemented a program that is a closed-loop control system whereby a motor that was being driven previsouly by velocity control is now modified using a position control system. A MATLAB tool was used to design an appropriate proportional-integral-derivative (PID) controller for the system.

1 Introduction

This laboratory exercise modified a previsouly implemented velocity control system to use position control instead. The closed-loop system is shown in Figure 1.

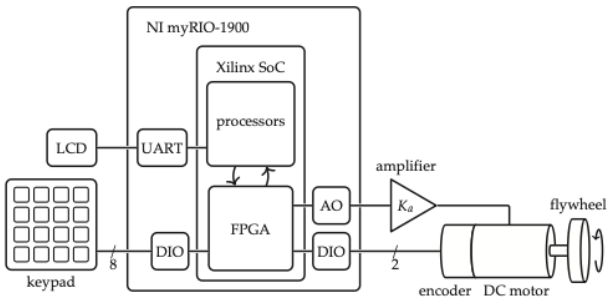

Figure 1: Schematic diagram of the target closed-loop system (Picone et al., 2024).

The objectives of this lab were:

1. To implement a position control system for an inertia-dominated load
2. To explore appropriate path planning
3. To integrate the use of a standard MATLAB design tool into the application development system

2 Materials and Methods

2.1 Materials

The materials used in this experiment were:

- Windows 10 computer with myRIO support for C and Eclipse IDE.
- NI myRIO-1900
- Keypad
- Serial display
- PMDC motor
- Analog current amplifier
- DC power supply
- Quadrature encoder
- Flywheel
- MATLAB (version R2026a Update 3 used)

2.2 PID Controller Design in MATLAB

MATLAB was used as the primary design environment to tune a proportional-integral-derivative (PID) controller suitable for the plant dynamics in this lab. The controller structure used for design and analysis is shown in Figure 2. For context, Figure 3 shows the control architecture from the previous lab (Lab 7), which was based on PI velocity control.

Comparing the two diagrams highlights the key change in this work: the PID/PIDF design introduces derivative action (with filtering) to improve transient shaping and robustness to measurement noise relative to the earlier PI-only formulation.

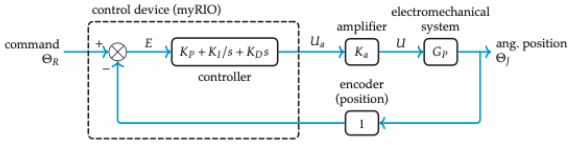

Figure 2: PID controller block diagram used for controller design (Picone et al., 2024).

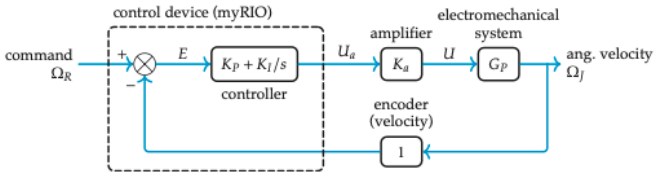

Figure 3: Control block diagram for the Lab 7 PI velocity-control system (Picone et al., 2024).

2.3 MATLAB Scripts

Two MATLAB scripts were developed for this portion of the lab: one for PID-family controller synthesis and one for performance evaluation against reference and model-based responses.

2.3.1 PID Controller Script

The first script was used to tune a PIDF compensator with MATLAB's Control System Toolbox using `pidtune()`. The design target was closed-loop reference tracking with an approximate control bandwidth of 10 Hz. A derivative-filtered PID (PIDF) form was selected to improve noise robustness by rolling off the high-frequency gain of the derivative action.

After obtaining the continuous-time controller model, the script converted it to a discrete-time implementation using `tf()`, `c2d()`, and `tfdata()` with sampling time $T = 0.005\,s$. The resulting discrete transfer function was then decomposed into second-order sections with `tf2sos()` to produce biquad realizations. Finally, `sos2header()` (Appendix E.1) exported those biquad coefficients to the C header file `myPIDF.h`, which was subsequently included by the program's `main()` routine.

2.3.2 Performance Analysis Script

The second script loaded the measured position-control response from `Lab8.mat` and compared it with two benchmarks: the ideal reference displacement trajectory and the response predicted by the dynamic model.

3 Laboratory Exploration and Evaluation

3.1 Problem L8.1

Write, test, and debug the program described in section L8.3.

The program can be found at https://github.com/juliansoh/ME477/blob/main/workspace/Julian_lab8/main.c.

The captured data in MATLAB file format can be found at https://github.com/juliansoh/ME477/blob/main/Reports/Lab8/data/Lab8.mat.

The program follows the same overall architecture used in Lab 7: a Main thread handled setup and user interaction through `ctable2()`, while a Timer thread handled interrupt-timed control execution and data logging. In this lab, the active controller definition was taken from the header generated by the MATLAB design script.

L8.3.1 Two Threads. Main Thread. The Main thread was responsible for initialization, thread setup, and operator interaction. Its flow was:

1. Initialize table-editor variables.
2. Initialize path-profile variables for ramp generation.
3. Configure the timer IRQ source (consistent with Labs 6 and 7).
4. Register and create the Timer thread, passing pointers to both the table data and the path profile through the thread-resource structure.
5. Launch the table editor with three display-only fields:
   - `P_ref`: revs
   - `P_act`: revs
   - `VDAout`: mV
6. On table-editor exit, clear the Timer-thread ready flag and wait for clean thread termination.

A representative resource structure was:

```
typedef struct {
  NiFpga_IrqContext irqContext; // context
  table *a_table;               // table
  seg *profile;                 // profile
  int nseg;                     // no. of segs
  NiFpga_Bool irqThreadRdy;     // ready flag
} ThreadResource;
```

Timer Thread. The Timer thread called the ISR-side logic each sample period. At thread start, shorthand pointers were created for table channels and ramp data, for example:

```
double *pref  = &((threadResource->a_table + 0)->
    value);
double *pact  = &((threadResource->a_table + 1)->
    value);
double *VDAmV = &((threadResource->a_table + 2)->
    value);
seg *mySegs = threadResource->profile;
int nseg = threadResource->nseg;
```

The thread ran in an IRQ-paced loop and exited only when its ready flag was cleared. Before entering the loop, it initialized analog I/O, forced the motor command to zero with `Aio_Write()`, and configured the encoder counter interface.

At each interrupt cycle, it performed the following:

1. Arm the next interval by waiting on IRQ assertion, writing the Timer Write Register, and writing `TRUE` to the Timer Set Time Register.
2. Call `Sramps()` to compute the current reference position Θ_R.
3. Call `pos()` to measure motor position Θ_J.
4. Compute control error as $e = \Theta_R - \Theta_J$.
5. Call `cascade()` with the PIDF filter to compute control voltage, then clamp the command to $[-10.0, +10.0]$ V.
6. Write the command to DAC channel AOCo via `Aio_Write()`.
7. Update table display values to reflect current controller state.
8. Log current-sample data for later analysis.

L8.4 Functions. Three functions were required inside the ISR loop:

- `cascade()`: reused the Lab 6 biquad difference-equation implementation (double-precision arithmetic throughout). For this application, one biquad section was used.
- `pos()`: read the encoder counter and returned displacement as `double` in BDI (encoder counts), referenced to the first recorded position.
- `Sramps()`: generated the reference trajectory Θ_R from a segment list, advancing one step per control cycle and repeating the full path continuously.

The path profile was initialized in `main()` and passed to the Timer thread using pointers in the shared resource structure. Access to the segment type required:

```
#include "sramps.h"

typedef struct {
  double xfa; // position
  double v;   // velocity limit
  double a;   // acceleration limit
  double d;   // dwell time (s)
} seg;
```

For Lab 8 testing, the profile used eight ramp segments:

```
double vmax = 50.0;
double amax = 20.0;
double dwell = 1.0;

seg mySegs[8] = {
  {10.125, vmax, amax, dwell},
  {20.250, vmax, amax, dwell},
  {30.375, vmax, amax, dwell},
  {40.500, vmax, amax, dwell},
  {30.625, vmax, amax, dwell},
  {20.750, vmax, amax, dwell},
  {10.875, vmax, amax, dwell},
  { 0.000, vmax, amax, dwell}
};
int nseg = 8;
```

This sequence produced four equal upward ramps (each 10.125 rev), followed by four downward ramps that returned the shaft to its starting position. All segments shared the same velocity and acceleration limits, with a 1 s dwell at each endpoint before progressing to the next segment.

The collected data were then opened in MATLAB from the `Lab8.mat` file, and the resulting plot is shown in Figure 4.

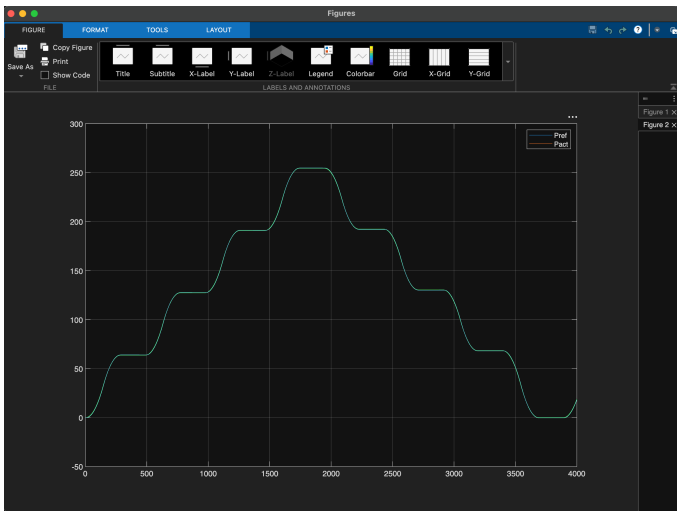

Figure 4: MATLAB plot generated from the collected data stored in `Lab8.mat`.

3.2 Problem L8.2

Write a MATLAB script (not the one in which you designed the controller) to analyze the result of running the program that you wrote in lab problem L8.1. It should do the following.

1. Load the experimental results from the Lab8.mat file.
2. Define a discrete version of the motor/load plant transfer from lab exercise 7. Consider using c2d().
3. Form the discrete controller from the values in the PIDF array in Lab8.mat.
4. Form the closed-loop system models relating the reference position Θ_R input to the position Θ_J and torque T outputs:

$$\frac{\Theta_J(z)}{\Theta_R(z)} = G_1(z), \qquad \frac{T(z)}{\Theta_R(z)} = G_2(z).$$

5. Using lsim(), simulate the system to find the theoretical responses for both the position $\Theta_J(t)$ and the torque $T(t)$ to the reference position $\Theta_R(t)$ array that you stored in Lab8.mat.
6. In a single MATLAB figure, plot the results in three subplot()s versus time, as follows:
   (a) Reference position, theoretical position, and experimental position.
   (b) Experimental error (reference-experimental position) and theoretical error (reference-theoretical position).
   (c) Theoretical and experimental torque.

The MATLAB script used to analyze the result of the running program in Problem L8.1 can be found at https://github.com/juliansoh/ME477/blob/main/Reports/Lab8/data/Lab8Analysis.mlx.

```
%% Lab 8 Analysis Script
% This script loads Lab8.mat, rebuilds the discrete plant and controller,
% simulates theoretical position and torque responses, and compares them
% to the experimental data.

clear;
clc;
close all;

%% 1. Load experimental data

load Lab8.mat

% Make sure data are column vectors
Pref = Pref(:);      % reference position, rad
Pact = Pact(:);      % experimental position, rad
Vout = Vout(:);      % experimental output voltage, V

% Rename sample time so T is not confused with torque
Ts = T(1);

% Time vector
N = length(Pref);
t = (0:N-1)' * Ts;

%% 2. Define continuous motor/load plant from Lab 7

% T1a parameters
Ka = 0.06;           % amplifier gain, A/V
Km = 0.0698;         % torque constant, N-m/A
J  = 7.8e-6;         % inertia, kg-m^2
B  = 0.0;            % damping, N-m-s/rad

```

```
% Voltage-to-position plant:
%
%               Ka*Km
% G(s) = ------------------
%        J*s^2 + B*s
%
% input: amplifier command voltage, V
% output: position, rad

Gs = tf(Ka*Km, [J B 0]);

%% 3. Convert plant to discrete time

Gz = c2d(Gs, Ts, 'zoh');

%% 4. Rebuild discrete controller from PIDF array

% PIDF was saved from C by casting the struct biquad
    array to double.
% Each biquad section has 11 doubles:
%
% b0 b1 b2 a0 a1 a2 x0 x1 x2 y1 y2
%
% Only the first six are coefficients.

PIDF = PIDF(:);

nsec = length(PIDF) / 11;

if abs(nsec - round(nsec)) > 0
    error('PIDF array length is not a multiple of
    11.');
end

nsec = round(nsec);

sections = reshape(PIDF, 11, nsec).';

Cz = tf(1, 1, Ts);

for k = 1:nsec
    b = sections(k, 1:3);
    a = sections(k, 4:6);

    Cz = Cz * tf(b, a, Ts);
end

%% 5. Form closed-loop systems

% Position model:
%
% G1(z) = Theta_J(z) / Theta_R(z)
%       = C(z)G(z) / (1 + C(z)G(z))

G1 = feedback(Cz * Gz, 1);

% Control-voltage model:
%
% U(z) / Theta_R(z) = C(z) / (1 + C(z)G(z))

G_u = feedback(Cz, Gz);

% Torque model:
%
% torque = Ka * Km * Vout
%
% G2(z) = Torque(z) / Theta_R(z)

G2 = (Ka * Km) * G_u;

%% 6. Simulate theoretical responses

Pact_theory = lsim(G1, Pref, t);

Torque_theory = lsim(G2, Pref, t);

%% 7. Compute experimental torque and errors

Torque_exp = Ka * Km * Vout;

Error_exp = Pref - Pact;
Error_theory = Pref - Pact_theory;

%% 8. Plot results

figure;

subplot(3,1,1)
plot(t, Pref, 'k', 'LineWidth', 1.2);
hold on;
plot(t, Pact_theory, 'b', 'LineWidth', 1.2);
plot(t, Pact, 'r', 'LineWidth', 1.0);
grid on;
xlabel('Time, s');
ylabel('Position, rad');
title('Lab 8 Position Response');
legend('Reference position', ...
  'Theoretical position', ...
  'Experimental position', ...
  'Location', 'best');

subplot(3,1,2)
plot(t, Error_exp, 'r', 'LineWidth', 1.0);
hold on;
plot(t, Error_theory, 'b', 'LineWidth', 1.2);
grid on;
xlabel('Time, s');
ylabel('Error, rad');
title('Position Error');
legend('Experimental error', ...
  'Theoretical error', ...
  'Location', 'best');

subplot(3,1,3)
plot(t, Torque_theory, 'b', 'LineWidth', 1.2);
hold on;
plot(t, Torque_exp, 'r', 'LineWidth', 1.0);
grid on;
xlabel('Time, s');
ylabel('Torque, N-m');
title('Motor Torque');
legend('Theoretical torque', ...
  'Experimental torque', ...
  'Location', 'best');

%% 9. Optional checks

disp('Loaded variables:')
whos

disp('Discrete controller Cz:')
Cz

disp('Discrete plant Gz:')
Gz

disp('Closed-loop position model G1:')
G1

disp('Closed-loop torque model G2:')
G2
```

The MATLAB output is shown in Figure 5. The three subplots are all shown in Figure 5:

- The top subplot shows reference position, theoretical position, and experimental position versus time.

- The middle subplot shows experimental error and theoretical error versus time.
- The bottom subplot shows theoretical torque and experimental torque versus time.

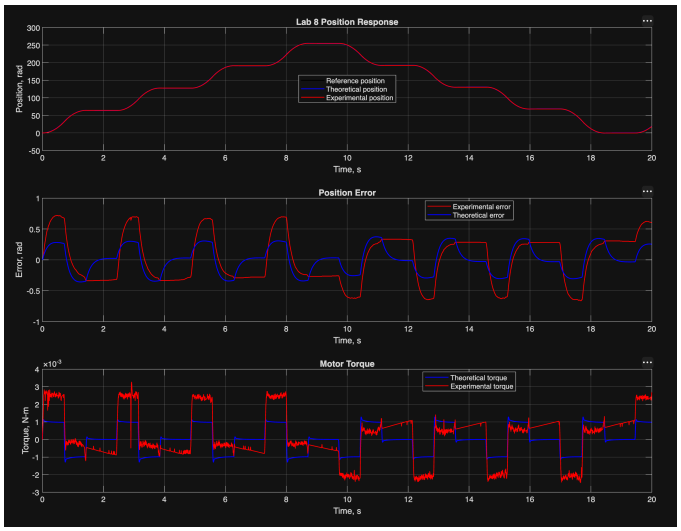

Figure 5: Lab 8 analysis results with three subplots: (a) position response, (b) position error, and (c) motor torque.

3.3 Problem L8.3

Analyze the results from lab problem L8.2. What do you conclude?

In analyzing the data, the following were observed:

1. Position Response The top plot in Figure 5 shows:

- Reference position (black)
- Theoretical position (blue)
- Experimental position (red)

The most important observation is that the red and blue curves almost completely overlap the reference trajectory.

Conclusion:

- The PIDF controller successfully tracked the commanded position profile.
- The actual motor position closely follows both the reference position and the theoretical prediction.
- There is little visible steady-state error.
- The motor follows both increasing and decreasing position commands correctly.
- The controller remained stable throughout the entire test. The closed-loop control behaved as expected, consistent with basic feedback-control principles where the controller continuously corrects the error between the reference and measured position.

2. Position Error The middle plot in Figure 5 shows:

- Experimental error (red)
- Theoretical error (blue)

Observations:

- The error remains relatively small compared to the overall position range.
- Peaks occur during acceleration and deceleration portions of the path.
- Error changes sign as the direction of motion reverses.
- Experimental error is consistently larger than theoretical error.

This is expected because the theoretical model assumes an ideal plant, while the real system contains effects that are difficult to model perfectly, such as:

- Friction
- Amplifier nonlinearities
- Encoder quantization
- Mechanical compliance
- Unmodeled disturbances

Conclusion:

- The theoretical model predicts the overall behavior well.
- The experimental system exhibits larger tracking errors than the ideal model.
- Despite these differences, the magnitude of the error remains small and bounded.
- The PIDF controller provides satisfactory position tracking performance.

3. Torque Response The bottom plot in Figure 5 shows:

- Theoretical torque (blue)
- Experimental torque (red)

Observations:

- Torque changes sign appropriately when the motor reverses direction.
- Large torque occurs during acceleration and deceleration events.
- Experimental torque follows the same overall trend as the theoretical prediction.
- Experimental torque contains noise and high-frequency variations that are not present in the theoretical model.

This difference is normal because the theoretical model cannot perfectly represent:

- Electrical noise
- Quantization effects
- Friction changes
- Mechanical disturbances

Conclusion:

- The experimental torque response agrees qualitatively with the theoretical prediction.
- The model correctly predicts when positive and negative torque are required.

- The experimental response includes additional noise and disturbances expected in a real physical system.

The Lab 8 PIDF position controller successfully tracked the commanded position trajectory. The experimental position response closely matched both the reference position and the theoretical model. Small tracking errors were observed during periods of acceleration, deceleration, and direction reversal, but these errors remained bounded and relatively small compared to the overall motion range. The experimental error was larger than the theoretical error because the mathematical model does not account for all real-world effects such as friction, sensor quantization, and other disturbances. The torque response followed the same general trend as the theoretical model, although the experimental torque exhibited additional noise. Overall, the results demonstrate that the designed PIDF controller provided stable and accurate position control for the motor-load system and that the analytical model gave a reasonable prediction of the system behavior.

4 Author Contributions

There was only one author for this report and lab.

References

Rico A. R. Picone, Joseph L. Garbini, and Cameron N. Devine. *An introduction to real-time computing for mechanical engineers*. The MIT Press, 2024.